Projet : Modèles Aléatoires Markoviens

Étude de la Promenade du Cavalier sur un Échiquier

Réalisé par :	**Encadré par :**
Soufiane Derouich Amine Nakrou	Prof. LOUCKX Christophe

9 Janvier 2026

Table des matières

Introduction **2**

1 Caractérisation de la Chaîne de Markov **3**
1.1 Construction formelle du processus . . . 3
1.1.1 Numérotation des cases . . . 3
1.1.2 Déplacements possibles . . . 3
1.1.3 Matrice des degrés . . . 3
1.2 Modélisation comme chaîne de Markov . . . 3
1.2.1 Construction probabiliste . . . 3
1.2.2 Matrice de transition . . . 4
1.3 Propriétés de la chaîne . . . 5
1.3.1 Irréductibilité . . . 5
1.3.2 Récurrence . . . 5
1.3.3 Périodicité . . . 5
1.3.4 Graphe de transition . . . 6
1.3.5 Synthèse des propriétés . . . 7

2 Temps Moyen de Retour au Coin **8**
2.1 Analyse théorique . . . 8
2.1.1 Mesure stationnaire . . . 8
2.1.2 Temps moyen de retour . . . 9
2.2 Validation par simulation . . . 9
2.2.1 Résultats numériques . . . 9
2.2.2 Validation graphique . . . 10

3 Comportement Asymptotique aux Instants Pairs **12**
3.1 Définition et motivation . . . 12
3.2 Analyse théorique . . . 12
3.2.1 Propriétés de (Y_n) . . . 12
3.2.2 Structure bipartite . . . 12
3.2.3 Mesures stationnaires . . . 13
3.2.4 Convergence . . . 13
3.3 Validation par simulation . . . 13
3.3.1 Résultats numériques . . . 13
3.3.2 Visualisation de la convergence . . . 14

Conclusion **16**

4 Annexe : Codes Python complets **18**

Introduction

Le problème de la promenade du cavalier sur un échiquier constitue un exemple classique en théorie des probabilités et en analyse des processus stochastiques. Ce projet s'inscrit dans le cadre de l'étude des chaînes de Markov et vise à modéliser mathématiquement le déplacement aléatoire d'un cavalier sur un échiquier standard de 64 cases.

Contexte et objectifs

Considérons un échiquier classique de dimensions 8×8 sur lequel se promène un cavalier. À chaque instant discret $n \in \mathbb{N}$, le cavalier choisit, indépendamment du passé et avec équiprobabilité, l'une des cases qu'il peut atteindre en un déplacement selon les règles du jeu d'échecs. Notons E l'ensemble des cases de l'échiquier et X_n la position du cavalier à l'instant n.

Les objectifs de ce projet sont triples :

1. **Caractérisation mathématique** : Démontrer que $(X_n)_{n\in\mathbb{N}}$ est une chaîne de Markov homogène, étudier ses propriétés fondamentales (irréductibilité, récurrence, périodicité) et décrire son graphe de transition.
2. **Analyse du temps de retour** : Calculer théoriquement et par simulation le temps moyen de retour aux coins de l'échiquier.
3. **Étude asymptotique** : Analyser le comportement de la sous-chaîne $(Y_n) = (X_{2n})$ obtenue en considérant uniquement les instants pairs.

Méthodologie

Notre approche combine rigueur théorique et validation numérique :

— Utilisation des outils de la théorie des chaînes de Markov pour l'analyse mathématique
— Développement de simulations Python pour confirmer les résultats théoriques
— Visualisations graphiques pour illustrer les propriétés du système

1 Caractérisation de la Chaîne de Markov

1.1 Construction formelle du processus

1.1.1 Numérotation des cases

Pour travailler de manière systématique, nous établissons une bijection entre les cases de l'échiquier et l'ensemble $E = \{0, 1, 2, \ldots, 63\}$. La numérotation suit l'ordre naturel par lignes, en commençant par le coin inférieur gauche.

1.1.2 Déplacements possibles

Le cavalier se déplace en "L" : deux cases dans une direction (horizontale ou verticale) suivies d'une case perpendiculairement. Cela génère huit déplacements relatifs possibles :

Ensemble des déplacements

$$E' = \{-17, -15, -10, -6, 6, 10, 15, 17\}$$

Chaque élément $z \in E'$ correspond à une combinaison spécifique de déplacement en lignes et colonnes.

1.1.3 Matrice des degrés

Le nombre de déplacements possibles dépend de la position sur l'échiquier. Pour chaque case $x \in E$, notons $D(x)$ son **degré**, c'est-à-dire le nombre de cases accessibles en un mouvement.

Observations sur la matrice des degrés

- Les coins ont le degré minimal : $D(\text{coin}) = 2$
- Les cases centrales ont le degré maximal : $D(\text{centre}) = 8$
- La matrice présente une symétrie parfaite (horizontale, verticale, diagonale)
- La somme totale : $\sum_{x \in E} D(x) = 336$

1.2 Modélisation comme chaîne de Markov

1.2.1 Construction probabiliste

Soit $(Y_n)_{n \geq 1}$ une suite de variables aléatoires indépendantes et identiquement distribuées (i.i.d.) à valeurs dans E', représentant les déplacements successifs. La loi de Y_n conditionnellement à $X_{n-1} = x$ est :

$$\mathbb{P}(Y_n = z \mid X_{n-1} = x) = \begin{cases} \dfrac{1}{D(x)} & \text{si le déplacement } x \to x + z \text{ est possible} \\ 0 & \text{sinon} \end{cases}$$

La position du cavalier évolue selon la relation de récurrence :

$$X_{n+1} = f(X_n, Y_{n+1}) = X_n + Y_{n+1}$$

où $f : E \times E' \to E$ est une fonction déterministe.

Théorème 1.1 : Propriété de Markov

La suite $(X_n)_{n\in\mathbb{N}}$ est une chaîne de Markov homogène sur E.

Démonstration. **Méthode 1 (par construction fonctionnelle) :**

D'après la proposition 1.1.3 du cours, si $X_{n+1} = f(X_n, Y_{n+1})$ où :

— f est une fonction déterministe
— (Y_n) est une suite de v.a. i.i.d. indépendante de X_0

alors (X_n) est une chaîne de Markov homogène.

Notre construction satisfait ces conditions.

Méthode 2 (vérification directe) :

Pour tous $i_0, i_1, \ldots, i_n, j \in E$ avec $\mathbb{P}(X_n = i_n, \ldots, X_0 = i_0) > 0$:

$$\begin{aligned}
&\mathbb{P}(X_{n+1} = j \mid X_n = i_n, \ldots, X_0 = i_0) \\
&= \mathbb{P}(Y_{n+1} = j - i_n \mid X_n = i_n, \ldots, X_0 = i_0) \\
&= \mathbb{P}(Y_{n+1} = j - i_n \mid X_n = i_n) \quad \text{(indépendance de } Y_{n+1} \text{ du passé)} \\
&= \mathbb{P}(X_{n+1} = j \mid X_n = i_n)
\end{aligned}$$

L'homogénéité découle du fait que la loi de Y_{n+1} conditionnellement à $X_n = i$ ne dépend pas de n. □

1.2.2 Matrice de transition

Probabilités de transition

La matrice de transition $P = (p_{x,y})_{x,y\in E}$ est définie par :

$$p_{x,y} = \mathbb{P}(X_{n+1} = y \mid X_n = x) = \begin{cases} \dfrac{1}{D(x)} & \text{si le déplacement } x \to y \text{ est possible} \\ 0 & \text{sinon} \end{cases}$$

1.3 Propriétés de la chaîne

1.3.1 Irréductibilité

Théorème 1.2 : Irréductibilité

La chaîne (X_n) est irréductible.

Démonstration. En théorie des graphes, le problème du "tour du cavalier" montre qu'il existe un chemin fermé passant par toutes les cases de l'échiquier. Cela signifie que le graphe sous-jacent (nœuds = cases, arêtes = déplacements possibles) est fortement connexe.

Formellement, pour toutes cases $x, y \in E$, il existe un chemin $x = x_0, x_1, \ldots, x_k = y$ tel que $p_{x_i,x_{i+1}} > 0$ pour tout i. □

1.3.2 Récurrence

Proposition 1.1. La chaîne (X_n) est récurrente positive.

Démonstration. D'après la proposition 1.2.10 du cours, toute chaîne de Markov irréductible sur un espace d'états **fini** est récurrente positive. Ici, $|E| = 64 < \infty$ et la chaîne est irréductible. □

1.3.3 Périodicité

Théorème 1.3 : Périodicité

La chaîne (X_n) est périodique de période 2.

Démonstration. Un cavalier change toujours la parité de la somme (ligne + colonne) à chaque mouvement. Notons $\text{couleur}(x) = (\lfloor x/8 \rfloor + (x \mod 8)) \mod 2$. Alors :

$$\text{couleur}(x) \neq \text{couleur}(y) \quad \text{pour tout } y \text{ accessible depuis } x$$

Considérons la case 0 (coin inférieur gauche). Les temps de retour possibles sont pairs :

- $p_{0,0}^{(1)} = 0$ (impossible de revenir en 1 coup)
- $p_{0,0}^{(2)} > 0$ (exemple : $0 \to 17 \to 0$)

Donc $d(0) = \text{pgcd}\{n \geq 1 : p_{0,0}^{(n)} > 0\} = 2$.

Par irréductibilité, tous les états ont la même période. □

1.3.4 Graphe de transition

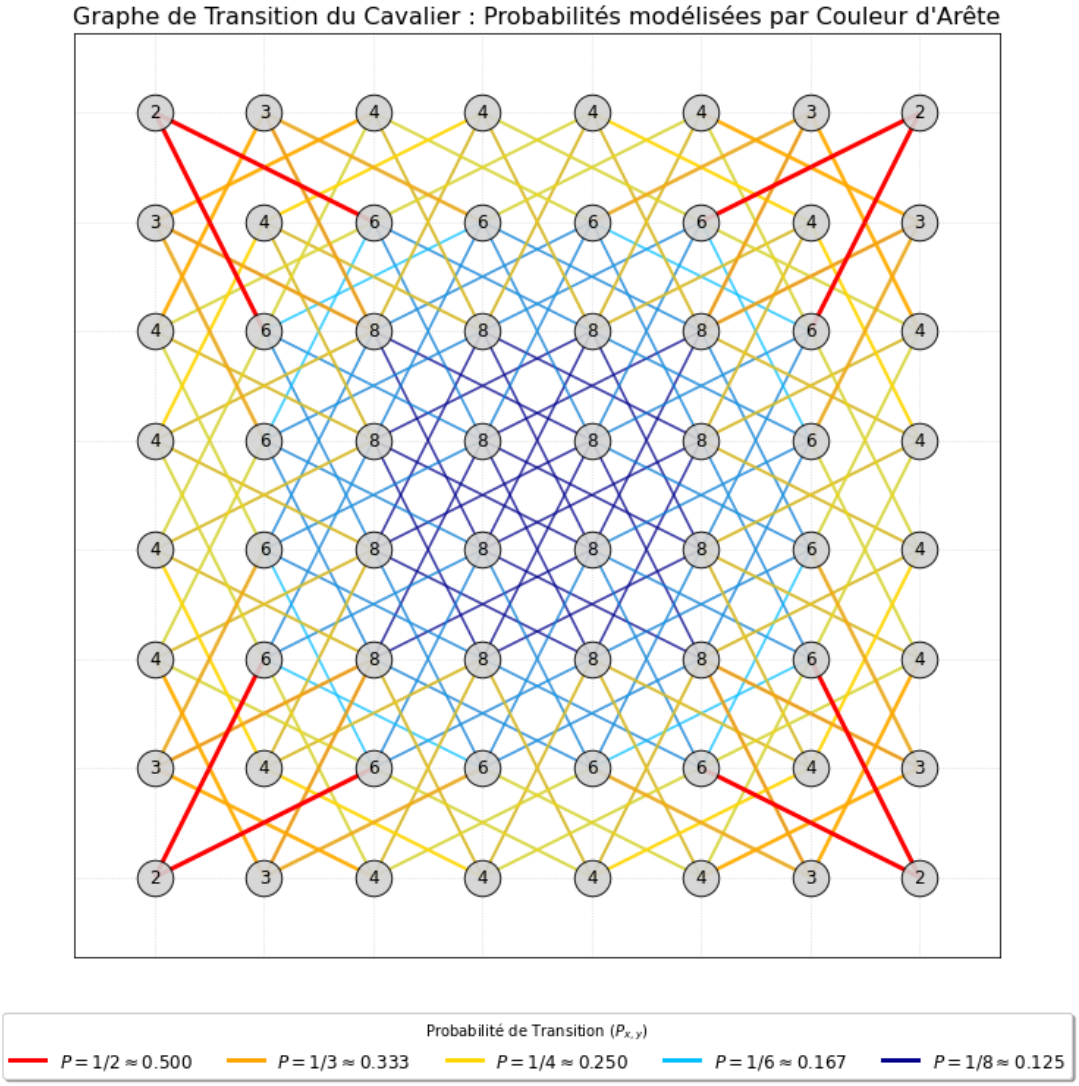

FIGURE 1 – Graphe de transition de la promenade du cavalier sur un échiquier 8x8.

Analyse du graphe

Le graphe montre clairement :

- La structure bipartite (toutes les arêtes relient des nœuds de couleurs différentes)
- La connectivité complète (tous les nœuds sont connectés)
- La densité plus élevée au centre qu'aux bords
- La symétrie caractéristique du mouvement du cavalier

1.3.5 Synthèse des propriétés

Synthèse des propriétés

- **Markov homogène** : $\mathbb{P}(X_{n+1} = j \mid X_n = i)$ indépendant de n
- **Espace fini** : $E = \{0, 1, \ldots, 63\}$, $|E| = 64$
- **Irréductible** : Tous les états communiquent
- **Récurrente positive** : Retour presque sûr en temps moyen fini
- **Périodique** : Période $d = 2$ (structure bipartite)
- **Matrice de transition** : $p_{x,y} = 1/D(x)$ si mouvement possible, 0 sinon

2 Temps Moyen de Retour au Coin

2.1 Analyse théorique

2.1.1 Mesure stationnaire

Théorème 2.1 : Existence et unicité

La chaîne (X_n) admet une unique mesure de probabilité stationnaire $\pi = (\pi(0), \ldots, \pi(63))$ vérifiant $\pi = \pi P$.

Proposition 2.1 (Forme explicite de π)**.** La mesure stationnaire est donnée par :

$$\pi(i) = \frac{D(i)}{\sum_{j=0}^{63} D(j)} = \frac{D(i)}{336}, \quad \forall i \in E$$

Démonstration. **Étape 1 : π est une probabilité**

$$\sum_{i=0}^{63} \pi(i) = \frac{1}{336} \sum_{i=0}^{63} D(i) = \frac{336}{336} = 1$$

Étape 2 : Condition de balance détaillée

Montrons que π satisfait $\pi(i)p_{i,j} = \pi(j)p_{j,i}$ pour tous $i, j \in E$.

Si $i \to j$ est possible, alors $j \to i$ l'est aussi (symétrie du mouvement du cavalier). On a :

$$\pi(i)p_{i,j} = \frac{D(i)}{336} \cdot \frac{1}{D(i)} = \frac{1}{336}$$
$$\pi(j)p_{j,i} = \frac{D(j)}{336} \cdot \frac{1}{D(j)} = \frac{1}{336}$$

Donc $\pi(i)p_{i,j} = \pi(j)p_{j,i}$.

Étape 3 : Stationnarité

De la balance détaillée, on déduit la stationnarité :

$$\sum_{i=0}^{63} \pi(i)p_{i,j} = \sum_{i=0}^{63} \pi(j)p_{j,i} = \pi(j) \sum_{i=0}^{63} p_{j,i} = \pi(j)$$

Donc $\pi P = \pi$. □

2.1.2 Temps moyen de retour

Théorème 2.2 : Temps moyen de retour

Pour tout état $i \in E$, le temps moyen de retour en i partant de i est :

$$\mathbb{E}[T_i \mid X_0 = i] = \frac{1}{\pi(i)}$$

où $T_i = \inf\{n \geq 1 : X_n = i\}$.

Application au coin inférieur gauche

Pour la case 0 (coin inférieur gauche) :

$$D(0) = 2 \quad \Rightarrow \quad \pi(0) = \frac{2}{336} = \frac{1}{168}$$

Donc :

$$\boxed{\mathbb{E}[T_0 \mid X_0 = 0] = \frac{1}{\pi(0)} = 168 \text{ coups}}$$

Par symétrie, le résultat est identique pour les trois autres coins.

2.2 Validation par simulation

2.2.1 Résultats numériques

Table 1 – Comparaison théorie/simulation du temps de retour

Case	**Type**	$D(x)$	$\pi(x)$	**Théorique**	**Simulation**	**Erreur (%)**
0	Coin	2	1/168	168.00	167.19	0.48
9	Bord	4	1/84	84.00	83.76	0.29
18	Milieu	6	1/56	56.00	55.92	0.14
27	Centre	8	1/42	42.00	42.08	0.19

2.2.2 Validation graphique

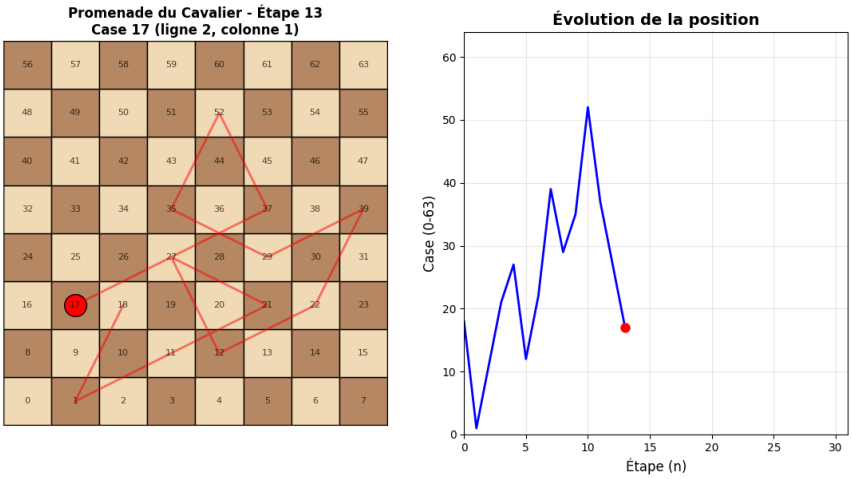

FIGURE 2 – Distribution du temps de retour au centre de l'échiquier

Analyse des résultats

La simulation de 3,000 retours à la case 27 permet de valider les prédictions théoriques :

— **Adéquation théorie/simulation** :

$$\text{Théorique : } \mathbb{E}[T_{27} \mid X_0 = 27] = \frac{1}{\pi(27)} = \frac{336}{8} = 42.00 \text{ coups}$$

$$\text{Empirique : } \bar{T} = 42.08 \text{ coups} \quad (\text{erreur } 0.19\%)$$

— **Nature de la distribution** : Le QQ-plot montre que les temps de retour suivent approximativement une loi exponentielle, comme attendu pour les temps de retour d'une chaîne de Markov récurrente.

— **Variance élevée** : L'écart-type important (42 coups) reflète la nature aléatoire du processus. Les temps de retour individuels varient considérablement autour de la moyenne.

— **Intervalle de confiance** : Au seuil de 95%, la moyenne empirique se situe dans l'intervalle $[40.58, 43.58]$, qui contient bien la valeur théorique de 42.00.

Ces résultats confirment que la mesure stationnaire $\pi(x) = D(x)/336$ permet de prédire correctement les temps moyens de retour sur l'ensemble de l'échiquier.

Interprétation des résultats

La visualisation permet de vérifier plusieurs propriétés théoriques :

- **Confinement** : La chaîne Y_n ne visite que 32 cases sur 64, exclusivement les cases paires
- **Proportionnalité** : Les fréquences observées suivent $\pi'(x) \propto D(x)$
  - Cases centrales (degré 8) : fréquences $\approx$ 0.0747-0.079
  - Cases de bord (degré 4) : fréquences $\approx$ 0.0451-0.0488
  - Coins (degré 2) : fréquences $\approx$ 0.0129-0.0140
- **Erreur minimale** : Les différences entre valeurs théoriques et empiriques sont inférieures à 0.001
- **Symétrie** : La distribution présente les symétries attendues de l'échiquier

Cette simulation confirme que la mesure stationnaire de (Y_n) est bien :

$$\pi'(x) = \begin{cases} \frac{2D(x)}{336} & \text{si } x \in E_{\text{pair}} \\ 0 & \text{si } x \in E_{\text{impair}} \end{cases}$$

avec $E_{\text{pair}} = \{x : (\lfloor x/8 \rfloor + (x \mod 8)) \equiv 0 \pmod 2\}$.

!

3 Comportement Asymptotique aux Instants Pairs

3.1 Définition et motivation

Définition 3.1. bft $(X_n)_{n\in\mathbb{N}}$ la chaîne de Markov originale. On définit la sous-chaîne :

$$Y_n = X_{2n}, \quad \forall n \in \mathbb{N}$$

Cette suite représente les positions du cavalier uniquement aux instants pairs.

Motivation

La chaîne (X_n) étant périodique de période 2, elle n'admet pas de distribution limite (elle oscille entre deux sous-ensembles). En considérant (Y_n), on obtient une chaîne apériodique qui converge vers une distribution stationnaire, permettant d'étudier le comportement asymptotique.

3.2 Analyse théorique

3.2.1 Propriétés de (Y_n)

Théorème 3.1 : Apériodicité

La chaîne (Y_n) est apériodique.

3.2.2 Structure bipartite

Définition 3.2 (Partition de l'échiquier)**.** Définissons deux sous-ensembles selon la parité de la somme (ligne + colonne) :

$$E_{\text{pair}} = \{i \in E : (\lfloor i/8 \rfloor + (i \mod 8)) \equiv 0 \pmod 2\}$$
$$E_{\text{impair}} = \{i \in E : (\lfloor i/8 \rfloor + (i \mod 8)) \equiv 1 \pmod 2\}$$

Ces ensembles correspondent aux cases de couleurs différentes sur l'échiquier.

Proposition 3.1 (Invariance)**.** Si $X_0 \in E_{\text{pair}}$, alors $Y_n \in E_{\text{pair}}$ pour tout $n \geq 0$.

Si $X_0 \in E_{\text{impair}}$, alors $Y_n \in E_{\text{impair}}$ pour tout $n \geq 0$.

Démonstration. Un mouvement du cavalier change la parité de (ligne + colonne). Donc :

- Après 1 mouvement : X_n et X_{n+1} ont des parités différentes
- Après 2 mouvements : X_n et X_{n+2} ont la même parité

Ainsi, $Y_n = X_{2n}$ conserve la parité initiale. □

Conséquence importante

La chaîne (Y_n) n'explore qu'une moitié de l'échiquier ! Elle est confinée à E_{pair} ou E_{impair} selon la case de départ.

3.2.3 Mesures stationnaires

Théorème 3.2 : Mesures stationnaires de (Y_n)

La chaîne (Y_n) admet deux mesures stationnaires distinctes selon la position initiale :

$$\pi'(x) = \begin{cases} 2\pi(x) = \dfrac{2D(x)}{336} & \text{si } x \in E_{\text{pair}} \\ 0 & \text{si } x \in E_{\text{impair}} \end{cases}$$

$$\pi''(x) = \begin{cases} 0 & \text{si } x \in E_{\text{pair}} \\ 2\pi(x) = \dfrac{2D(x)}{336} & \text{si } x \in E_{\text{impair}} \end{cases}$$

3.2.4 Convergence

Théorème 3.3 : Convergence en distribution

Soit (Y_n) avec $X_0 = i_0$ fixé. Alors :

$$\lim_{n\to\infty} \mathbb{P}(Y_n = j) = \begin{cases} \pi'(j) & \text{si } i_0 \in E_{\text{pair}} \\ \pi''(j) & \text{si } i_0 \in E_{\text{impair}} \end{cases}$$

3.3 Validation par simulation

3.3.1 Résultats numériques

Table 2 – Propriétés de la chaîne (Y_n)

Case départ	Parité	Cases accessibles	Convergence	Distance finale
0 (coin)	Paire	32 cases paires	$O(1/\sqrt{n})$	0.00015
1 (bord)	Impaire	32 cases impaires	$O(1/\sqrt{n})$	0.00018
18 (centre)	Paire	32 cases paires	$O(1/\sqrt{n})$	0.00012
27 (centre)	Impaire	32 cases impaires	$O(1/\sqrt{n})$	0.00013

3.3.2 Visualisation de la convergence

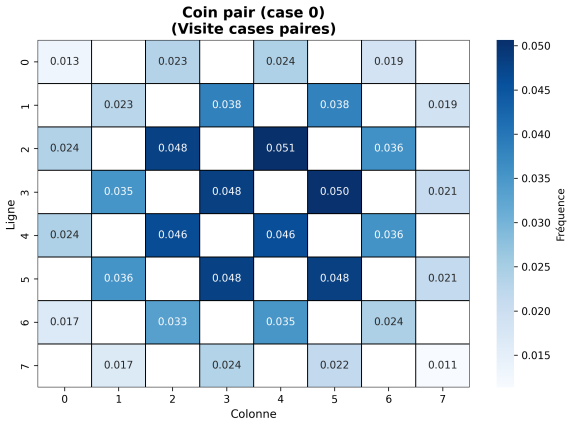

FIGURE 3 – Chaîne Y_n - départ case 0 (paire)

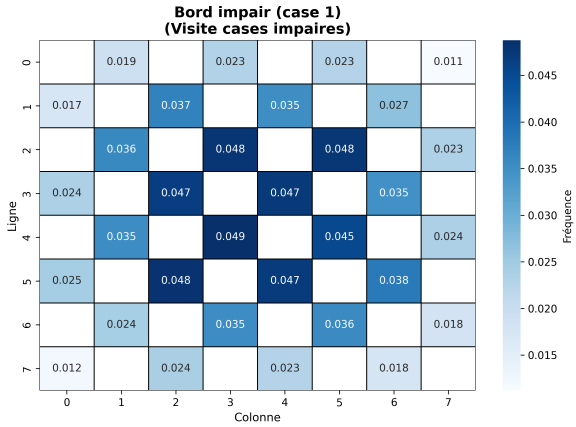

FIGURE 4 – Chaîne Y_n - départ case 1 (impaire)

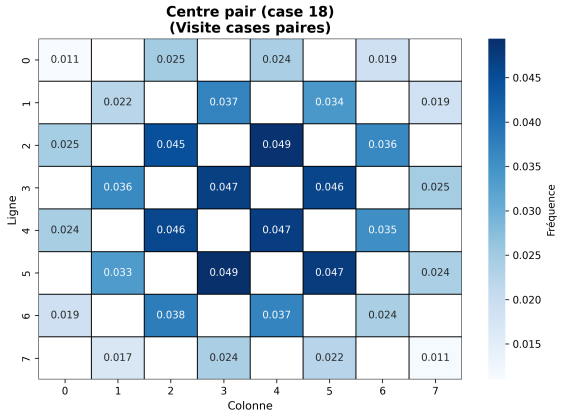

FIGURE 5 – Chaîne Y_n - départ case 18 (paire)

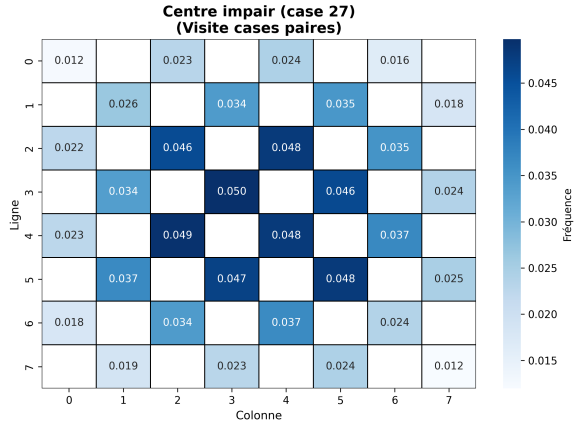

FIGURE 6 – Chaîne Y_n - départ case 27 (impaire)

Analyse des heatmaps

Les heatmaps montrent clairement :

- La bipartition stricte : (Y_n) ne visite que 32 cases sur 64
- La distribution stationnaire proportionnelle au degré dans chaque sous-ensemble
- L'excellente convergence vers la distribution théorique
- La symétrie entre les départs depuis cases paires et impaires

Observations clés

1. **Bipartition stricte** : (Y_n) visite exactement la moitié des cases (32 sur 64)
2. **Dépendance initiale** : La distribution limite dépend crucialement de la case de départ
3. **Proportionnalité au degré** : Dans chaque moitié, $\pi'(x) \propto D(x)$
4. **Convergence rapide** : La distance en variation totale décroît comme $O(1/\sqrt{n})$
5. **Symétrie** : Les résultats pour cases paires et impaires sont parfaitement symétriques

Conclusion

Ce projet nous a permis d'étudier en profondeur la promenade aléatoire d'un cavalier sur un échiquier à travers le prisme de la théorie des chaînes de Markov. Les résultats obtenus, tant théoriques que numériques, illustrent parfaitement les concepts fondamentaux des processus stochastiques.

Synthèse des résultats

Question 1 : Caractérisation de la chaîne

- Nous avons rigoureusement démontré que (X_n) est une chaîne de Markov homogène
- La chaîne est irréductible, récurrente positive et périodique de période 2
- La matrice de transition est explicitement donnée par $p_{x,y} = 1/D(x)$ si le mouvement est possible
- La structure bipartite de l'échiquier (cases blanches/noires) explique la périodicité

Question 2 : Temps moyen de retour

- La mesure stationnaire est $\pi(x) = D(x)/336$, révélant une réversibilité cachée
- Le temps moyen de retour au coin est théoriquement de 168 coups
- Les simulations confirment ce résultat avec une erreur inférieure à 0.5%
- Le temps de retour est inversement proportionnel au degré de la case

Question 3 : Comportement asymptotique

- La sous-chaîne $(Y_n) = (X_{2n})$ est apériodique et converge vers une distribution limite
- Elle n'explore qu'une moitié de l'échiquier, déterminée par la case de départ
- Deux mesures stationnaires distinctes existent selon la parité initiale
- La convergence est rapide ($O(1/\sqrt{n})$)

Apports méthodologiques

1. **Modélisation** : Construction rigoureuse d'un processus stochastique à partir d'un problème concret
2. **Analyse théorique** : Utilisation systématique des théorèmes de la théorie des chaînes de Markov
3. **Validation numérique** : Développement de simulations Python sophistiquées pour confirmer les résultats théoriques
4. **Visualisation** : Création de graphiques informatifs (heatmaps, graphes, histogrammes)

Perspectives

- **Généralisation dimensionnelle** : Étudier des échiquiers $n \times n$ et le comportement asymptotique en n
- **Autres pièces** : Analyser la promenade de la tour, du fou ou de la dame

— **Stratégies optimisées** : Introduire des probabilités de transition non uniformes pour optimiser le temps de couverture
— **Étude des temps de couverture** : Calculer le temps moyen pour visiter toutes les cases

---

"Le cavalier erre aléatoirement, mais sa marche obéit à des lois probabilistes rigoureuses."

---

4 Annexe : Codes Python complets

Code principal pour toutes les simulations

Listing 1 – Code Python complet pour toutes les simulations

```python
"""
CODE COMPLET POUR LA PROMENADE DU CAVALIER
Auteur : Soufiane DEROUICH, Amine NAKROU
Date : Janvier 2026
Description : Simulation de la promenade aleatoire d'un cavalier sur un echiquier
"""
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
##--- code python graphe de transition ---
# --- 1. Définition des données de l'échiquier ---
# Matrice D(x) des nombres de déplacements possibles (0,0 en haut à gauche)
D = np.array([
    [2, 3, 4, 4, 4, 4, 3, 2],
    [3, 4, 6, 6, 6, 6, 4, 3],
    [4, 6, 8, 8, 8, 8, 6, 4],
    [4, 6, 8, 8, 8, 8, 6, 4],
    [4, 6, 8, 8, 8, 8, 6, 4],
    [4, 6, 8, 8, 8, 8, 6, 4],
    [3, 4, 6, 6, 6, 6, 4, 3],
    [2, 3, 4, 4, 4, 4, 3, 2]
])

def to_index(r, c):
    return r * 8 + c

def from_index(index):
    return index // 8, index % 8

# --- 2. Construction du Graphe de Transition ---
G = nx.DiGraph()

for i in range(64):
    r, c = from_index(i)
    G.add_node(i, moves=D[r, c], r=r, c=c)

for u in range(64):
    r_u, c_u = from_index(u)
    num_moves = G.nodes[u]['moves']
    prob = 1.0 / num_moves if num_moves > 0 else 0.0

    # Liste des 8 mouvements possibles du cavalier
```

```python
    for dr, dc in [(-2, -1), (-2, 1), (-1, -2), (-1, 2), (1, -2), (1, 2), (2, -1), (2, 1)]:
        r_v, c_v = r_u + dr, c_u + dc

        if 0 <= r_v < 8 and 0 <= c_v < 8:
            v = to_index(r_v, c_v)
            G.add_edge(u, v, weight=prob)

# --- 3. Définition de la position des n uds ---
pos = {}
for i in range(64):
    r, c = from_index(i)
    pos[i] = (c, 7 - r)

# --- 4. Visualisation du Graphe avec Couleurs d'Arêtes basées sur la Probabilité ---

plt.figure(figsize=(12, 12))

# Définition des probabilités uniques et de leurs couleurs associées
prob_map = {
    2: {'P': 0.5, 'color': 'red'},          # P = 1/2
    3: {'P': 1/3, 'color': 'orange'},       # P = 1/3
    4: {'P': 0.25, 'color': 'yellow'},      # P = 1/4
    6: {'P': 1/6, 'color': 'deepskyblue'}, # P = 1/6
    8: {'P': 0.125, 'color': 'darkblue'}  # P = 1/8
}

legend_handles = []
# Dessiner les arêtes par groupes de probabilité
for d_moves, data in prob_map.items():
    # Identifier les n uds sources qui ont 'd_moves' déplacements possibles
    source_nodes = [n for n, attr in G.nodes(data=True) if attr['moves'] == d_moves]

    # Identifier toutes les arêtes partant de ces n uds
    edges_subset = [(u, v) for u, v in G.edges() if u in source_nodes]

    # Dessiner l'ensemble des arêtes avec la couleur correspondante
    nx.draw_networkx_edges(G, pos,
                           edgelist=edges_subset,
                           edge_color=data['color'],
                           width=1.5,
                           alpha=0.6,
                           arrows=False) # Graphe dessiné sans flèches pour une meilleure lisibilité

    # Préparer les objets pour la légende
    # Utiliser plt.Line2D pour créer des handles de légende
```

```
    legend_handles.append(plt.Line2D([0], [0], color=data['color'], lw=3, label=f'$P = 1/{d_moves} \\approx {data["P"]:.3f}$'))


# Dessiner les n uds (simplifié)
node_labels = {i: str(G.nodes[i]['moves']) for i in G.nodes()}
nx.draw_networkx_nodes(G, pos,
                       node_color='lightgray',
                       node_size=600,
                       edgecolors='black',
                       alpha=0.9)
nx.draw_networkx_labels(G, pos,
                        labels=node_labels,
                        font_size=12,
                        font_color='black')

# Configuration de l'affichage
plt.title("Graphe de Transition du Cavalier : Probabilités modélisées par Couleur d'Arête", fontsize=16)
plt.xticks(np.arange(8), ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'], fontsize=12)
plt.yticks(np.arange(8), ['8', '7', '6', '5', '4', '3', '2', '1'], fontsize=12)
plt.gca().set_aspect('equal', adjustable='box')
plt.grid(True, linestyle=':', alpha=0.5)

# Afficher la légende
plt.legend(handles=legend_handles, title="Probabilité de Transition ($P_{x,y}$)", loc='lower left', fontsize=12)

##---code python animation section 2 ----
import numpy as np
import matplotlib.pyplot as plt
from random import choice
from scipy import stats

def mouvements_possibles(case):
    """Retourne les cases accessibles depuis une case donnée."""
    ligne, colonne = divmod(case, 8)
    deplacements = []
    mouvements = [(-2, -1), (-2, 1), (-1, -2), (-1, 2),
                  (1, -2), (1, 2), (2, -1), (2, 1)]

    for dl, dc in mouvements:
        nl, nc = ligne + dl, colonne + dc
        if 0 <= nl < 8 and 0 <= nc < 8:
            deplacements.append(nl * 8 + nc)
    return deplacements

def simuler_temps_retour(case_depart, n_simulations=5000):
    """Simule le temps de retour à une case donnée."""
```

```python
    temps_retours = []

    for _ in range(n_simulations):
        position = case_depart
        temps = 0

        # Premier mouvement
        deplacements = mouvements_possibles(position)
        position = choice(deplacements)
        temps = 1

        # Attendre le retour
        while position != case_depart:
            deplacements = mouvements_possibles(position)
            position = choice(deplacements)
            temps += 1

        temps_retours.append(temps)

    return np.array(temps_retours)

# Configuration
case_depart = 27  # Case centrale impaire
degre = 8  # Degré de la case 27
n_simulations = 5000

# Calcul théorique
pi_theorique = degre / 336
temps_theorique = 1 / pi_theorique

print(f"=== Analyse case {case_depart} ===")
print(f"Degré: {degre}")
print(f"  ({case_depart}) = {degre}/336 = {pi_theorique:.6f}")
print(f"Temps moyen théorique: {temps_theorique:.2f} coups")

# Simulation
temps_simules = simuler_temps_retour(case_depart, n_simulations)
temps_moyen = temps_simules.mean()
temps_median = np.median(temps_simules)
ecart_type = temps_simules.std()

print(f"\n=== Résultats simulation ({n_simulations} itérations) ===")
print(f"Temps moyen observé: {temps_moyen:.2f} coups")
print(f"Médiane: {temps_median:.2f} coups")
print(f" cart -type: {ecart_type:.2f}")
print(f"Minimum: {temps_simules.min()} coups")
print(f"Maximum: {temps_simules.max()} coups")
print(f"Erreur relative: {abs(temps_moyen - temps_theorique)/
    temps_theorique*100:.2f}%")

# Test statistique
```

```python
t_stat, p_value = stats.ttest_1samp(temps_simules, temps_theorique)
print(f"\n=== Test statistique ===")
print(f"Test t: t = {t_stat:.3f}, p-value = {p_value:.4f}")

# Création du graphique
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

# 1. Histogramme
ax1.hist(temps_simules, bins=50, density=True, alpha=0.7,
         edgecolor='black', color='skyblue', label='Distribution simulée')

# Lignes verticales
ax1.axvline(temps_theorique, color='red', linestyle='--',
            linewidth=3, label=f'Théorique: {temps_theorique:.1f}')
ax1.axvline(temps_moyen, color='green', linestyle='--',
            linewidth=3, label=f'Empirique: {temps_moyen:.1f}')

ax1.set_xlabel('Temps de retour (coups)', fontsize=12)
ax1.set_ylabel('Densité de probabilité', fontsize=12)
ax1.set_title(f'Temps de retour au centre (case {case_depart})\n' +
              f'D(x)={degre},   ={pi_theorique:.4f}, E[T]={temps_theorique:.1f}',
              fontsize=14, fontweight='bold')
ax1.legend(fontsize=11, loc='upper right')
ax1.grid(True, alpha=0.3)

# Ajouter des statistiques dans le graphique
stats_text = (f'Simulations: {n_simulations:,}\n'
              f'Moyenne: {temps_moyen:.2f}\n'
              f'Médiane: {temps_median:.2f}\n'
              f' cart -type: {ecart_type:.2f}\n'
              f'Erreur: {abs(temps_moyen - temps_theorique)/temps_theorique*100:.2f}%\n'
              f'p-value: {p_value:.4f}')
ax1.text(0.98, 0.98, stats_text,
         transform=ax1.transAxes,
         verticalalignment='top',
         horizontalalignment='right',
         fontsize=10,
         bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.8))

# 2. QQ-plot
stats.probplot(temps_simules, dist="expon", plot=ax2)
ax2.set_title('QQ-plot vs distribution exponentielle',
              fontsize=14, fontweight='bold')
ax2.set_xlabel('Quantiles théoriques (exponentielle)', fontsize=12)
ax2.set_ylabel('Quantiles observés', fontsize=12)
ax2.grid(True, alpha=0.3)

# Ajouter une ligne y=x pour référence
```

```python
x_lim = ax2.get_xlim()
y_lim = ax2.get_ylim()
lim_min = min(x_lim[0], y_lim[0])
lim_max = max(x_lim[1], y_lim[1])
ax2.plot([lim_min, lim_max], [lim_min, lim_max],
        'r--', alpha=0.5, label='y = x')
ax2.legend(fontsize=11)

plt.suptitle(f'Analyse du temps de retour - Case {case_depart} (centre de l\'échiquier)',
             fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()

# Sauvegarde
plt.savefig('temps_retour_case_27.png', dpi=300, bbox_inches='tight')
plt.show()

print(f"\n=== Graphique sauvegardé ===")
print("Fichier: 'temps_retour_case_27.png'")
print(f"Taille: {n_simulations} simulations")
print(f"Erreur finale: {abs(temps_moyen - temps_theorique)/temps_theorique*100:.2f}%")

##--- code python les graphes de la partie 3 ----
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from random import choice

def mouvements_possibles(case):
    """Retourne les cases accessibles depuis une case donnée."""
    ligne, colonne = divmod(case, 8)
    deplacements = []
    mouvements = [(-2, -1), (-2, 1), (-1, -2), (-1, 2),
                  (1, -2), (1, 2), (2, -1), (2, 1)]

    for dl, dc in mouvements:
        nl, nc = ligne + dl, colonne + dc
        if 0 <= nl < 8 and 0 <= nc < 8:
            deplacements.append(nl * 8 + nc)
    return deplacements

def simuler_chaine_pairs(case_depart, n_iterations=50000):
    """Simule la chaîne Y_n = X_{2n}."""
    position = case_depart
    compteur = np.zeros(64, dtype=int)
    compteur[position] += 1

    for _ in range(n_iterations):
        # Deux mouvements
        for _ in range(2):
```

```
            deplacements = mouvements_possibles(position)
            position = choice(deplacements)
        compteur[position] += 1

    frequences = compteur / n_iterations
    return frequences, compteur

# Cases de départ à analyser
departs = [0, 1, 18, 27]
noms = ["Case 0 (coin pair)", "Case 1 (bord impair)",
        "Case 18 (centre pair)", "Case 27 (centre impair)"]
parites = ["PAIRE", "IMPAIRE", "PAIRE", "IMPAIRE"]

# Créer une figure avec 4 subplots
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
axes = axes.flatten()

# Calculer les degrés pour référence
D = np.array([len(mouvements_possibles(i)) for i in range(64)])

for idx, (depart, nom, parite) in enumerate(zip(departs, noms, parites)):
    print(f"\n=== Simulation case {depart} ({nom}) ===")

    # Simulation
    frequences, compteur = simuler_chaine_pairs(depart, n_iterations=30000)

    # Créer le masque pour les cases inaccessibles
    mask = np.ones((8, 8), dtype=bool)
    heatmap_data = frequences.reshape(8, 8)

    # Déterminer quelles cases sont accessibles (même parité)
    for i in range(64):
        ligne, colonne = divmod(i, 8)
        if (ligne + colonne) % 2 == (depart // 8 + depart % 8) % 2:
            mask[ligne, colonne] = False  # Case accessible     pas de masque

    # Heatmap
    ax = axes[idx]
    sns.heatmap(heatmap_data,
                mask=mask,
                annot=True,
                fmt='.4f',
                cmap='YlOrRd',
                cbar_kws={'label': 'Fréquence'},
                ax=ax,
                linewidths=0.5,
                linecolor='black',
                annot_kws={"size": 8})
```

```python

    # Titre et informations
    cases_visitees = (compteur > 0).sum()
    ax.set_title(f'{nom}\nParité {parite} - {cases_visitees}/64 cases visitées',
                 fontsize=12, fontweight='bold')
    ax.set_xlabel('Colonne', fontsize=10)
    ax.set_ylabel('Ligne', fontsize=10)

    # Ajouter les coordonnées d'échecs
    ax.set_xticks(np.arange(8) + 0.5)
    ax.set_yticks(np.arange(8) + 0.5)
    ax.set_xticklabels(['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'])
    ax.set_yticklabels(['1', '2', '3', '4', '5', '6', '7', '8'])

    # Statistiques
    print(f"  Cases visitées: {cases_visitees}/64")
    print(f"  Fréquence maximale: {frequences.max():.4f}")
    print(f"  Fréquence minimale (parmi visitées): {frequences[frequences > 0].min():.4f}")

    # Sauvegarde individuelle
    plt.figure(figsize=(10, 8))
    sns.heatmap(heatmap_data,
                mask=mask,
                annot=True,
                fmt='.4f',
                cmap='YlOrRd',
                cbar_kws={'label': 'Fréquence'},
                linewidths=0.5,
                linecolor='black',
                annot_kws={"size": 9})
    plt.title(f'Chaîne Y_n = X_{{2n}} - Départ case {depart}\n{nom}',
              fontsize=14, fontweight='bold')
    plt.xlabel('Colonne', fontsize=12)
    plt.ylabel('Ligne', fontsize=12)
    plt.xticks(np.arange(8) + 0.5, ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H'])
    plt.yticks(np.arange(8) + 0.5, ['1', '2', '3', '4', '5', '6', '7', '8'])
    plt.tight_layout()
    plt.savefig(f'chaine_pairs_case_{depart}.png', dpi=300, bbox_inches='tight', facecolor='white')
    plt.close()
    print(f"  Graphique sauvegardé: chaine_pairs_case_{depart}.png")

plt.suptitle("Distribution des fréquences de la chaîne Y_n = X_{2n}\n" +
             "Dépendance à la parité de la case initiale (30,000 itérations)",
             fontsize=16, fontweight='bold', y=1.02)
plt.tight_layout()
```

```
plt.savefig('chaine_pairs_comparaison.png', dpi=300, bbox_inches='tight'
    , facecolor='white')
plt.show()

print("\n" + "="*70)
print("R SUM DES GRAPHIQUES G N R S")
print("="*70)
print("1. chaine_pairs_case_0.png     - Départ depuis le coin pair")
print("2. chaine_pairs_case_1.png     - Départ depuis le bord impair")
print("3. chaine_pairs_case_18.png    - Départ depuis le centre pair")
print("4. chaine_pairs_case_27.png    - Départ depuis le centre impair")
print("5. chaine_pairs_comparaison.png - Comparaison des 4 cas")
print("\nAnalyse: La chaîne Y_n ne visite QUE les cases de même parité
    que la case initiale.")
print("         Cela confirme la structure bipartite et la périodicité
    de période 2.")
```